IS THE GAP AN INTERFACE GAP? AN AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM DEFINITION AND MOTIVATION

Data practitioners working with data lakehouses spend a significant portion of their time answering three types of questions: which table should I use, and can I trust it; how was this table or metric derived; and which tables do I need to join to answer my question. Today the first is answered by keyword search in a catalog such as Amundsen or Unity Catalog, ranked by popularity (Grover, 2019), or by asking colleagues (Koesten et al., 2017). The second is answered by a lineage view or by opening the dbt model and reading the SQL (Das, 2020). The third is answered by finding a query someone else already wrote (Databricks, 2026b) or by relying on familiarity with the data which takes considerable time to acquire. What makes answering all three types of questions hard and time-consuming is the same thing: the information needed is split across the catalog's metadata, the pipeline code and lineage that produced the tables, and the data itself, and data practitioners have to interface with all three by hand.

Answering these questions is well suited for agents because the task requires performing a complex chain of lookups and making sense of large amounts of data with tool calls (Koesten et al., 2017). Practitioners already use agents this way: coding agents such as Claude Code, Codex CLI, and OpenCode can be pointed at a catalog through an MCP server that lists and describes tables, fetches lineage, and runs SQL (Databricks, 2025), and vendors sell purpose-built data agents such as Databricks Genie and Snowflake Cortex Analyst (Databricks, 2026a; Snowflake, 2024). Both, however, fall short of what practitioners need.

The gaps are of three kinds. First, answers come back without evidence, so a user cannot tell whether the agent read the SQL or guessed from a table name. Second, the catalog tools available to general agents expose metadata search, lineage, natural-language-to-SQL over curated views, and SQL execution (Databricks, 2025; DataHub, 2026; Snowflake, 2026), but none breaks a table's defining code into its sources, filters, and joins or offers cheap, budgeted probes of the data. An agent must therefore infer how a table was built from names and comments that are often stale (Meta, 2025), or reconstruct it through open-ended queries that add cost without guaranteeing a correct answer. Third, accuracy is low: Databricks reports a leading coding agent with its own MCP servers solving 32% of real tasks (Databricks, 2026a).

The problem this project addresses is therefore to identify which component of an agentic question-answering system accounts for the gap between generic and specialized agents, and to improve that component at the lowest cost. A generic coding agent and a specialized data agent differ in three components at once: the *tools* the agent is given, the *harness* (the program around the model that feeds it the conversation, runs its tool calls, manages context, and decides when to stop), and the *model*. I hypothesize that the tools are the component that matters: that a better *catalog-computer interface*—a tool layer that exposes the metadata, the pipeline code and lineage, and the data in a form an agent can use, with evidence attached to every result—will yield the largest improvement in accuracy per unit of cost, for any harness and especially for cheaper models. I measure this on *catalog-grounded question answering*: given one of the three question types above (selection, derivation, joins), return the answer with its evidence or say that no trustworthy answer exists within a fixed cost budget.

2 COMPARISON WITH EXISTING WORK

Prior work on this problem falls into three groups: data catalogs, agentic systems, and agent evaluation. Each addresses part of the problem but leaves a gap.

Catalogs such as Amundsen and DataHub pair a metadata graph with search, lineage, and popularity signals (Halevy et al., 2016; Grover, 2019), but a search still returns a ranked list rather than a

recommendation, descriptions go stale faster than people can maintain them (Meta, 2025), and a deprecated copy of a table is as easy to find as the good one (Lyft, 2019). Semantic layers and natural-language-to-SQL products such as Snowflake Cortex Analyst are accurate inside a curated slice of data (Snowflake, 2024), but someone has to manually curate that slice first.

Among agentic systems, vendor data agents such as Databricks Genie report much higher accuracy than a coding agent given generic catalog tools, but because these systems are closed, no one can isolate which component produced the gain (Databricks, 2026a). Research agents and benchmarks over data lakes, such as the Metadata Reasoner and LakeQA (Zhang et al., 2026; Wang et al., 2026), show that an agent can select the right tables from metadata but stop at selection: none reads pipeline code to explain a derivation or reports cost across model sizes.

On evaluation, Terminal-Bench found that the model matters more than the harness (Merrill et al., 2026), but every harness it tested had the same key tool, a shell, so it never tested whether better tools matter more, as I expect they do for data catalogs. SWE-agent showed that better tools alone raise accuracy, but only in a single coding harness (Yang et al., 2024); whether better catalog tools raise accuracy across harnesses and let cheaper models match frontier ones is unknown. Prior harness comparisons vary the harness and model (Kapoor et al., 2026; Laban et al., 2026) but never the tools, so how much of an agent's accuracy per dollar comes from its tools remains unmeasured.

3 Next Steps and Direction

To test my hypothesis that the tool has the biggest impact on accuracy per unit of cost, I will build a new catalog-computer interface, give it to existing harnesses in place of their generic tools, and measure changes in performance on a set of evaluation tasks.

The catalog-computer interface will be a Python package served over either MCP or as a command-line tool, so any harness can use it unmodified. It will be built on the open-source Unity Catalog server's REST API (Unity Catalog, 2026), DuckDB for queries, the dbt manifest for pipeline structure, and `sqlglot` for parsing SQL. For the metadata, the interface will offer filtered search and compact table descriptions; for the pipeline, the interface will expose the ability to derive column lineages, as well as the sources, filters, joins, and aggregations, from SQL; and for the data, the interface will provide cheap checks such as row counts, key uniqueness, null rates, and date ranges. Every result will carry an evidence ID that the agent will be instructed to cite.

This project will also include building a lakehouse because derivation and join questions need a pipeline layer that public datasets lack. A Docker Compose stack will run the Unity Catalog server, Iceberg tables, DuckDB, and a dbt project whose models and metric definitions will form the pipeline layer. The data will come from TPC-DS (Nambiar & Poess, 2006) and NYC Open Data (City of New York, 2026), and it will be made deliberately messy with duplicated and deprecated tables, stale comments, and undocumented columns (Zhang et al., 2026).

The evaluation set will contain 180 questions in four categories: table selection, derivation, joins, and verification. Examples include "which table has daily store-level sales for 2025 broken down by SKU, and is it the canonical one" (selection), "how is `net_revenue` computed, and which filters exclude refunds" (derivation), "which tables, joined on which keys, give revenue by customer cohort and region" (joins), and "is `c_customer_sk` unique in `customer`" (verification).

To run the comparisons, I will build a driver that runs each agent harness from a script rather than its interactive interface, with every harness connected to the same tool server. Where supported, the driver will use the Agent Client Protocol (Zed Industries, 2025), a standard interface for sending prompts to any coding agent and reading back its responses, so that one driver can control every harness. The harnesses I plan to try are mini-SWE-agent as a minimal baseline, OpenCode, DeepSeek Harness, Codex CLI, and Claude Code (Lieret et al., 2025; DeepSeek AI, 2026). The models I plan to try span sizes within each family and both closed (e.g., Claude Haiku, Sonnet, and Opus) and open-weight (e.g., Qwen at several sizes, and DeepSeek's Flash and Pro models). Every supported harness-model pair will be run on each question three times with the generic tools and three times with mine. For each run, I will record whether the answer was correct and evidenced, its dollar cost, and its tool calls. These measurements will show how much my interface raises the performance of each harness, how much harnesses differ given the same tools, and ultimately whether a small model with it can match a frontier model without it.

REFERENCES

City of New York. NYC open data, 2026. URL https://opendata.cityofnewyork.us.

Shirshanka Das. DataHub: Popular metadata architectures explained. LinkedIn Engineering Blog, 2020. URL https://engineering.linkedin.com/blog/2020/datahub-popular-metadata-architectures-explained.

Databricks. Announcing managed MCP servers with Unity Catalog and Mosaic AI integration. Databricks Blog, 2025. URL https://www.databricks.com/blog/announcing-managed-mcp-servers-unity-catalog-and-mosaic-ai-integration.

Databricks. Introducing Genie Code. Databricks Blog, March 11, 2026, 2026a. URL https://www.databricks.com/blog/introducing-genie-code.

Databricks. View table insights and popularity in Catalog Explorer; AI-generated comments. Databricks documentation, 2026b. URL https://docs.databricks.com/en/data/table-insights.html.

DataHub. DataHub MCP server. DataHub documentation, 2026. URL https://docs.datahub.com/docs/features/feature-guides/mcp.

DeepSeek AI. DeepSeek Harness (dsh): An open-source agent runtime. GitHub repository deepseek-ai/deepseek-harness (MIT), developer preview released August 13, 2026, 2026. URL https://github.com/deepseek-ai/deepseek-harness.

Mark Grover. Amundsen—Lyft's data discovery & metadata engine. Lyft Engineering Blog, 2019. URL https://eng.lyft.com/amundsen-lyfts-data-discovery-metadata-engine-62d27254fbb9.

Alon Halevy, Flip Korn, Natalya F. Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. Goods: Organizing Google's datasets. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, 2016.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.

Laura M. Koesten, Emilia Kacprzak, Jenifer F. A. Tennison, and Elena Simperl. The trials and tribulations of working with structured data—a study on information seeking behaviour. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pp. 1277–1289, 2017.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. Outstanding Paper; arXiv:2505.06120.

Kilian Lieret, John Yang, et al. mini-SWE-agent: The 100-line AI agent that solves GitHub issues. GitHub repository SWE-agent/mini-swe-agent (MIT), 2025. URL https://github.com/SWE-agent/mini-swe-agent.

Lyft. Disrupting data discovery with Amundsen. Talks by Mark Grover, Tao Feng, and Daniel Won at Strata SF and Data Council SF, March 2019; slides report ∼30% of data-scientist time spent on discovery and a 75% reduction, 2019. URL https://www.datacouncil.ai/hubfs/DataEngConf/Data%20Council/Slides%20SF%2019/Amundsen%20-%20A%20Data%20Discovery%20Platform%20from%20Lyft.pdf.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2601.11868.

Meta. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/.

Raghunath Othayoth Nambiar and Meikel Poess. The making of TPC-DS. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*, pp. 1049–1058, 2006.

Snowflake. Snowflake Cortex Analyst: Behind the scenes. Snowflake Engineering Blog, August 14, 2024, 2024. URL https://www.snowflake.com/en/blog/engineering/snowflake-cortex-analyst-behind-the-scenes/.

Snowflake. Snowflake-managed MCP server. Snowflake documentation, 2026. URL https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-agents-mcp.

Unity Catalog. Unity Catalog: Open, multimodal catalog for data & AI (Apache-2.0). GitHub repository and REST API specification, 2026. URL https://github.com/unitycatalog/unitycatalog.

Haonan Wang, Jiaxiang Liu, Yurong Liu, Austin Senna Wijaya, Tianle Zhou, Eden Wu, Yijia Chen, Wanting You, Reya Vir, Daniela Pinto, Grace Fan, Yusen Zhang, Juliana Freire, and Eugene Wu. LakeQA: An exploratory QA benchmark over a million-scale data lake. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.10460.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zed Industries. Agent client protocol (ACP): A protocol for connecting editors and AI agents. Zed external-agents documentation: https://zed.dev/docs/ai/external-agents, 2025. URL https://agentclientprotocol.com.

Jiani Zhang, Sercan Ö. Arık, Cosmin Arad, Fatma Özcan, and Alon Halevy. An agentic approach to metadata reasoning. *arXiv preprint arXiv:2604.20144*, 2026. v2, August 2026.

A AI-USE DISCLOSURE STATEMENT

In this work, I used generative AI tools to refine the hypothesis, to brainstorm task formulations, and to give feedback on the research methodology. The remaining required-disclosure tasks (generating or cleaning datasets, mathematical claims or proofs, implementing methods, translation, qualitative data analysis, and interpreting results) are not applicable given that this is just a proposal rather than the final study. Additionally, I used generative AI tools for brainstorming, identifying gaps in prior work, searching for and summarizing relevant literature, and providing editing suggestions for readability. I have reviewed all AI-assisted work, including the literature summaries, for accuracy, and I take responsibility for the final content of this work, including text, claims, or artifacts produced with the aid of generative AI.